PRACTICAL LABORATORY ASSIGNMENT

Full Name	OSAWARU PRAISE
Registration Number	C11/26/CGRCE/17564
Date Submitted	11th October 2026
Course	GRC 102 Week 5
Linux Distribution Used	Kali Linux

https://github.com/PraiseOsawaru/ICDFA-Assignments/tree/main/GRC_102/GRC102_W5_Lab

Part 1: Setting Up a Simulated Environment (30 points)

1.

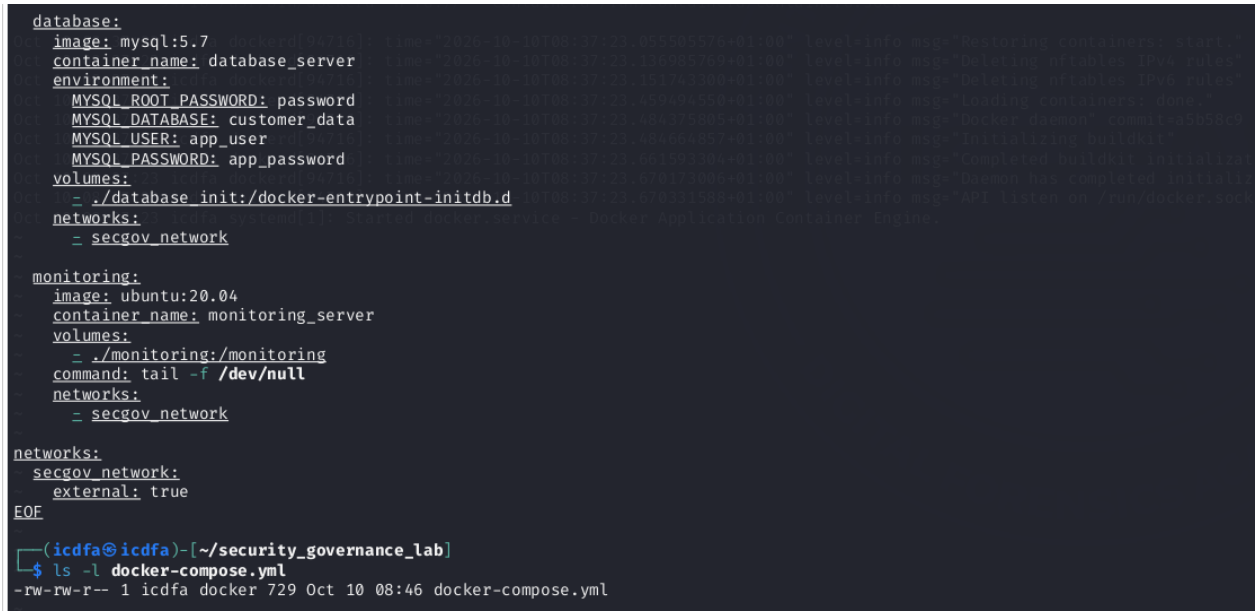


Figure 1 Screenshots showing the Docker environment setup

2.

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat > initialize_governance.py << 'EOF'
#!/usr/bin/env python3
from governance_tracker import GovernanceTracker

def initialize_governance_data():
    tracker = GovernanceTracker()
    p_id = tracker.add_policy("Patch Management Policy", "Policy governing timely patches", "CISO", "2023-01-15
", "2024-01-15")
    tracker.add_control("Patch Management Process", "Process for identifying patches", p_id, "IT Manager", "202
3-01-20")
    tracker.add_metric("Patch Compliance", "Percentage of systems patched", 95.0, 85.0, "May 2023", "Improving"
)
    print("Governance tracker initialized with sample data.")

if __name__ == "__main__":
    initialize_governance_data()
EOF
chmod +x initialize_governance.py
./initialize_governance.py
Governance tracker initialized with sample data.

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ./vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261010_095522.json
Summary: 3 vulnerabilities found
  Critical: 0
  High: 0
  Medium: 1
```

Figure 2 Vulnerability Scan report

3. Governance Tracker Dashboard

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ mkdir -p database_init
cat > database_init/init.sql << 'EOF'
USE customer_data;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state, zip, email, phone, credit_card)
VALUES
  ('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA', '12345', 'john.doe@example.com',
 '555-123-4567', '4111111111111111'),
  ('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere', 'NY', '67890', 'jane.smith@example
.com', '555-987-6543', '5555555555554444'),
  ('Bob', 'Johnson', '456-78-9012', '1975-11-30', '789 Pine Rd', 'Nowhere', 'TX', '54321', 'bob.johnson@example
.com', '555-456-7890', '378282246310005');
EOF

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ ls -l database_init/init.sql
-rw-rw-r-- 1 icdfa docker 896 Oct 10 08:53 database_init/init.sql
```

Figure 3 Governance tracker dashboard

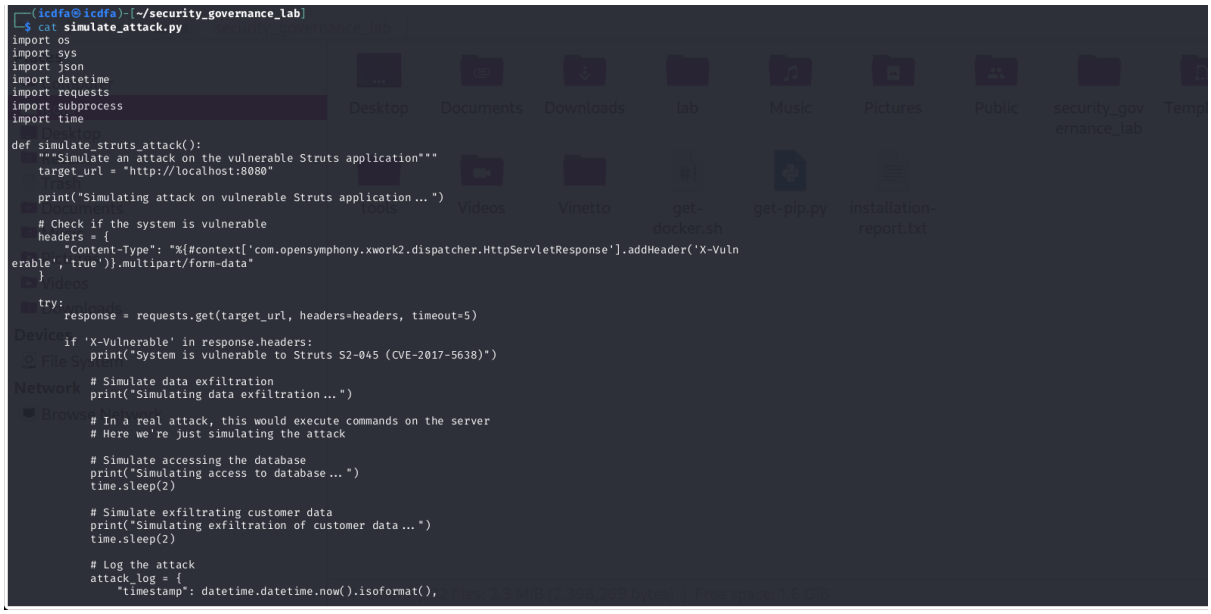
3. A brief explanation (1-2 paragraphs) of how this simulated environment reflects the conditions that led to the Equifax breach

The simulated environment directly models the conditions that led to the 2017 Equifax data breach by deploying a vulnerable version of Apache Struts (CVE-2017-5638) on the web server without the necessary security updates. Just like the real-world incident where a known critical vulnerability went unaddressed for months, this setup leaves the primary application entry point exposed to exploitation and remote code execution.

Additionally, the lab setup replicates crucial architectural and access control failures identified in the Equifax case study, such as insufficient network segmentation that permits direct access from the web server to the database server, alongside weak database authentication using default credentials. Together, these unmitigated vulnerabilities and gaps in policy implementation create a controlled environment where simulated attacks can easily bypass security layers and compromise sensitive customer data.

Part 2: Analyzing Security Governance Failures

1.



```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat simulate_attack.py
import os
import sys
import json
import datetime
import requests
import subprocess
import time

def simulate_struts_attack():
    """Simulate an attack on the vulnerable Struts application"""
    target_url = "http://localhost:8080"

    print("Simulating attack on vulnerable Struts application...")

    # Check if the system is vulnerable
    headers = {
        "Content-Type": "%{#context['com.opensymphony.xwork2.dispatcher.HttpServletResponse'].addHeader('X-Vuln
erable','true')}.multipart/form-data"
    }

    try:
        response = requests.get(target_url, headers=headers, timeout=5)

        if 'X-Vulnerable' in response.headers:
            print("System is vulnerable to Struts S2-045 (CVE-2017-5638)")

            # Simulate data exfiltration
            print("Simulating data exfiltration...")

            # In a real attack, this would execute commands on the server
            # Here we're just simulating the attack

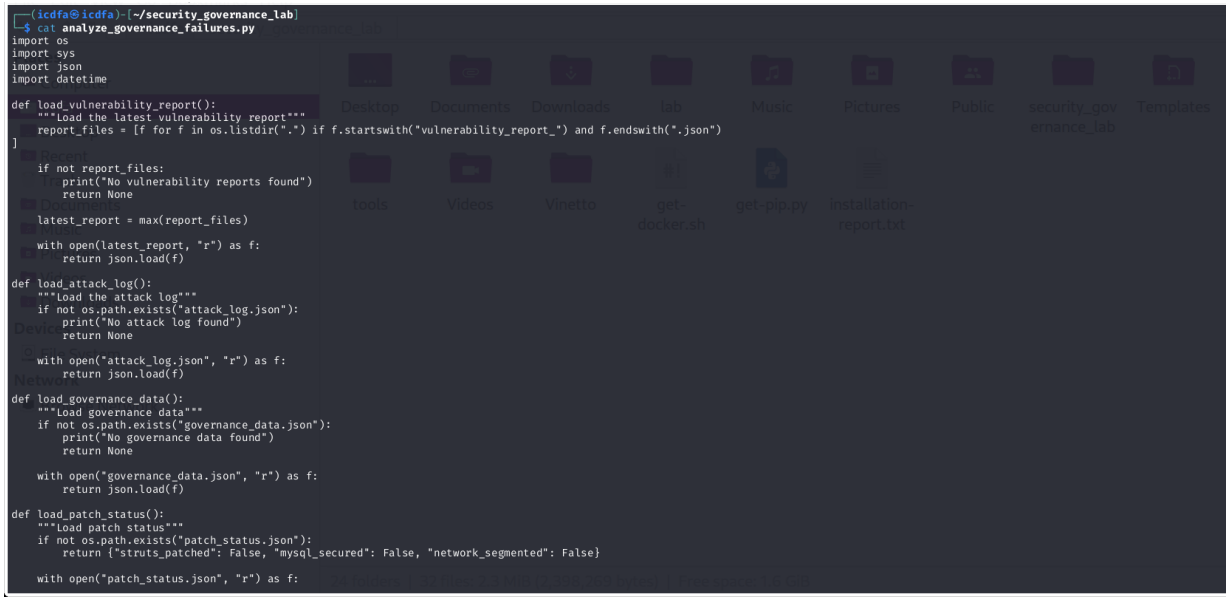
            # Simulate accessing the database
            print("Simulating access to database...")
            time.sleep(2)

            # Simulate exfiltrating customer data
            print("Simulating exfiltration of customer data...")
            time.sleep(2)

            # Log the attack
            attack_log = {
                "timestamp": datetime.datetime.now().isoformat(),
```

Figure 4 Attack simulation log

2.



```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat analyze_governance_failures.py
import os
import sys
import json
import datetime

def load_vulnerability_report():
    """Load the latest vulnerability report"""
    report_files = [f for f in os.listdir(".") if f.startswith("vulnerability_report_") and f.endswith(".json")
]

    if not report_files:
        print("No vulnerability reports found")
        return None

    latest_report = max(report_files)

    with open(latest_report, "r") as f:
        return json.load(f)

def load_attack_log():
    """Load the attack log"""
    if not os.path.exists("attack_log.json"):
        print("No attack log found")
        return None

    with open("attack_log.json", "r") as f:
        return json.load(f)

def load_governance_data():
    """Load governance data"""
    if not os.path.exists("governance_data.json"):
        print("No governance data found")
        return None

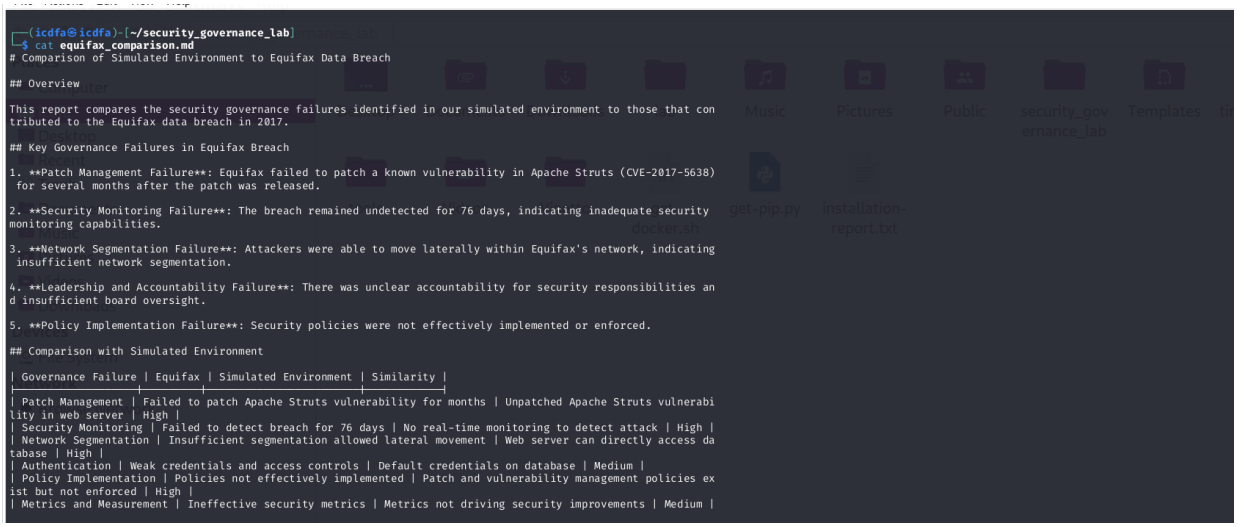
    with open("governance_data.json", "r") as f:
        return json.load(f)

def load_patch_status():
    """Load patch status"""
    if not os.path.exists("patch_status.json"):
        return {"struts_patched": False, "mysql_secured": False, "network_segmented": False}

    with open("patch_status.json", "r") as f:
```

Figure 5 Governance failure analysis report

3.



```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat equifax_comparison.md
# Comparison of Simulated Environment to Equifax Data Breach

## Overview

This report compares the security governance failures identified in our simulated environment to those that con
tributed to the Equifax data breach in 2017.

## Key Governance Failures in Equifax Breach

1. **Patch Management Failure**: Equifax failed to patch a known vulnerability in Apache Struts (CVE-2017-5638)
 for several months after the patch was released.

2. **Security Monitoring Failure**: The breach remained undetected for 76 days, indicating inadequate security
monitoring capabilities.

3. **Network Segmentation Failure**: Attackers were able to move laterally within Equifax's network, indicating
 insufficient network segmentation.

4. **Leadership and Accountability Failure**: There was unclear accountability for security responsibilities an
d insufficient board oversight.

5. **Policy Implementation Failure**: Security policies were not effectively implemented or enforced.

## Comparison with Simulated Environment

| Governance Failure | Equifax | Simulated Environment | Similarity |
|---|---|---|---|
| Patch Management | Failed to patch Apache Struts vulnerability for months | Unpatched Apache Struts vulnerabi
lity in web server | High |
| Security Monitoring | Failed to detect breach for 76 days | No real-time monitoring to detect attack | High |
| Network Segmentation | Insufficient segmentation allowed lateral movement | Web server can directly access da
tabase | High |
| Authentication | Weak credentials and access controls | Default credentials on database | Medium |
| Policy Implementation | Policies not effectively implemented | Patch and vulnerability management policies ex
ist but not enforced | High |
| Metrics and Measurement | Ineffective security metrics | Metrics not driving security improvements | Medium |
```

Figure 6 The comparison report between the simulated environment and the Equifax case study

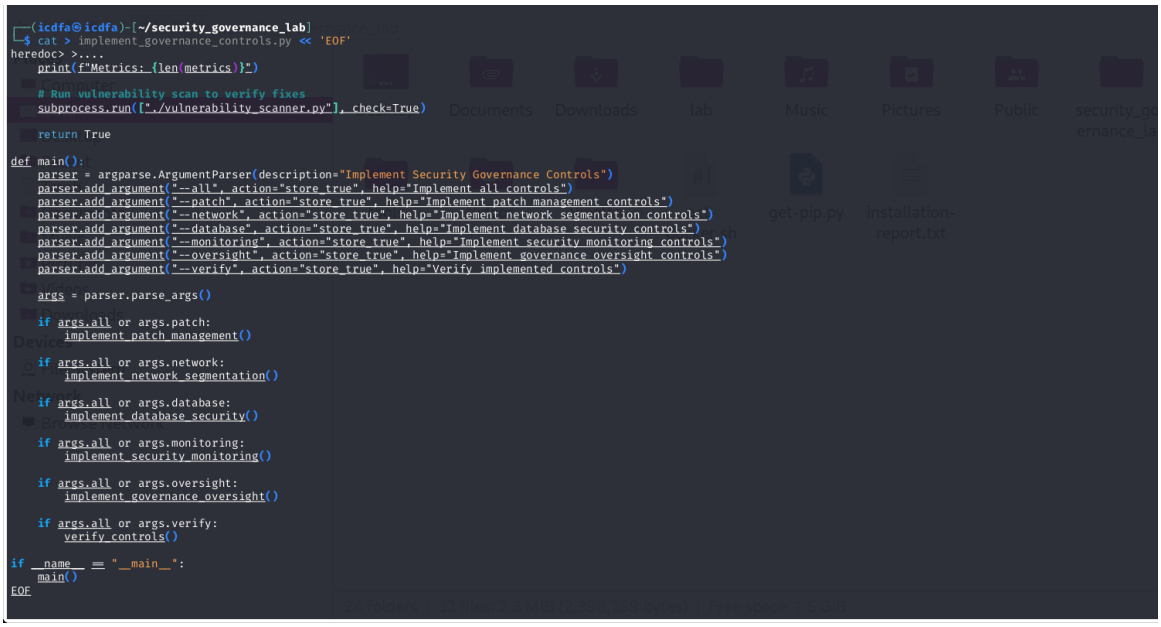
4. A brief explanation (1-2 paragraphs) of how this simulated environment reflects the conditions that led to the Equifax breach

The primary lesson from this analysis is that having written security policies is entirely ineffective if they are not actively implemented, enforced, and audited. Both the Equifax breach and our simulated environment demonstrate how critical gaps such as delaying patch deployment for high-severity vulnerabilities like Apache Struts or relying on default credentials can persist despite formal governance documentation. Without clear operational ownership, automated oversight, and strict compliance enforcement, policies remain passive guidelines rather than protective measures.

Additionally, the analysis underscores the necessity of continuous monitoring and defense-in-depth architecture to contain security failures. When perimeter defenses fail, strong internal controls such as network segmentation, restrictive access rights, and real-time incident detection prevent an initial web application exploit from escalating into widespread database compromise and data exfiltration. Security governance must therefore drive actionable metrics that actively measure control effectiveness, ensure timely risk remediation, and provide executive oversight to prevent catastrophic failures.

3. Part 3 Implementing Security Governance Controls

1.



```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat > implement_governance_controls.py << 'EOF'
heredoc> >....
    print(f"Metrics: {len(metrics)}")

    # Run vulnerability scan to verify fixes
    subprocess.run(["./vulnerability_scanner.py"], check=True)

    return True

def main():
    parser = argparse.ArgumentParser(description="Implement Security Governance Controls")
    parser.add_argument("--all", action="store_true", help="Implement all controls")
    parser.add_argument("--patch", action="store_true", help="Implement patch management controls")
    parser.add_argument("--network", action="store_true", help="Implement network segmentation controls")
    parser.add_argument("--database", action="store_true", help="Implement database security controls")
    parser.add_argument("--monitoring", action="store_true", help="Implement security monitoring controls")
    parser.add_argument("--oversight", action="store_true", help="Implement governance oversight controls")
    parser.add_argument("--verify", action="store_true", help="Verify implemented controls")

    args = parser.parse_args()

    if args.all or args.patch:
        implement_patch_management()

    if args.all or args.network:
        implement_network_segmentation()

    if args.all or args.database:
        implement_database_security()

    if args.all or args.monitoring:
        implement_security_monitoring()

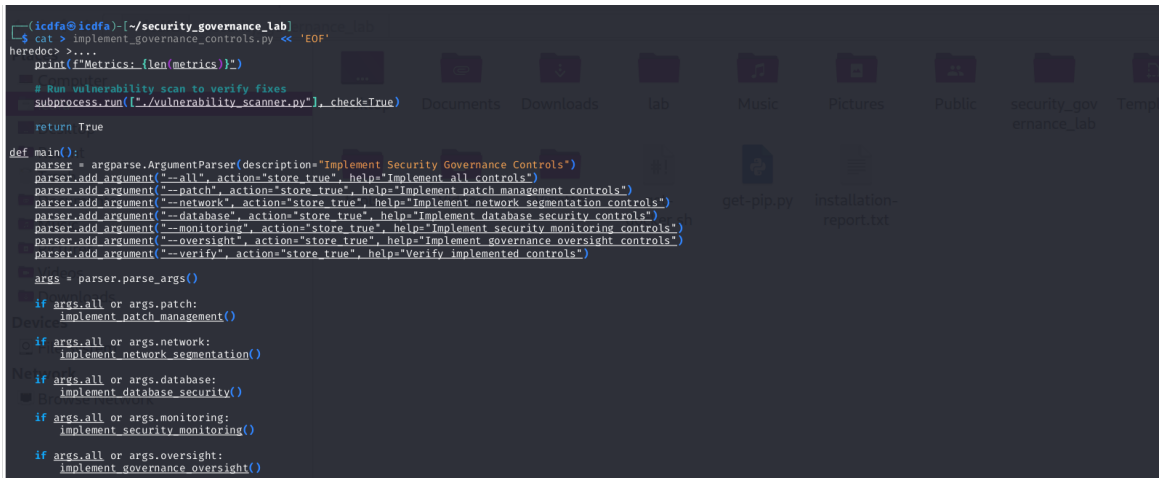
    if args.all or args.oversight:
        implement_governance_oversight()

    if args.all or args.verify:
        verify_controls()

if __name__ == "__main__":
    main()
EOF
```

Figure 7 Governance control implementation script

2.



```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat > implement_governance_controls.py << 'EOF'
heredoc> >....
    print(f"Metrics: {len(metrics)}")

    # Run vulnerability scan to verify fixes
    subprocess.run(["./vulnerability_scanner.py"], check=True)

    return True

def main():
    parser = argparse.ArgumentParser(description="Implement Security Governance Controls")
    parser.add_argument("--all", action="store_true", help="Implement all controls")
    parser.add_argument("--patch", action="store_true", help="Implement patch management controls")
    parser.add_argument("--network", action="store_true", help="Implement network segmentation controls")
    parser.add_argument("--database", action="store_true", help="Implement database security controls")
    parser.add_argument("--monitoring", action="store_true", help="Implement security monitoring controls")
    parser.add_argument("--oversight", action="store_true", help="Implement governance oversight controls")
    parser.add_argument("--verify", action="store_true", help="Verify implemented controls")

    args = parser.parse_args()

    if args.all or args.patch:
        implement_patch_management()

    if args.all or args.network:
        implement_network_segmentation()

    if args.all or args.database:
        implement_database_security()

    if args.all or args.monitoring:
        implement_security_monitoring()

    if args.all or args.oversight:
        implement_governance_oversight()
```

Figure 8 Governance controls report

3. A brief explanation (1-2 paragraphs) of how the implemented controls address the governance failures identified in the Equifax case study

The implemented security governance controls directly resolve the core patch management and vulnerability tracking failures that caused the Equifax breach by instituting automated patch application and verifying critical fixes such as updating Apache Struts. By enforcing strict compliance targets and establishing clear operational accountability, organizations can prevent known vulnerabilities from lingering unaddressed for months.

Additionally, the introduction of network segmentation, robust database authentication, and real-time security monitoring eliminates the architectural blind spots that allowed attackers to

move laterally and exfiltrate data undetected in the Equifax incident. Combined with structured executive and board-level oversight policies, these technical and administrative controls ensure that security mandates are actively enforced and continuously audited to mitigate systemic risk

Part 4: Developing Security Governance Metrics and Reporting

1.

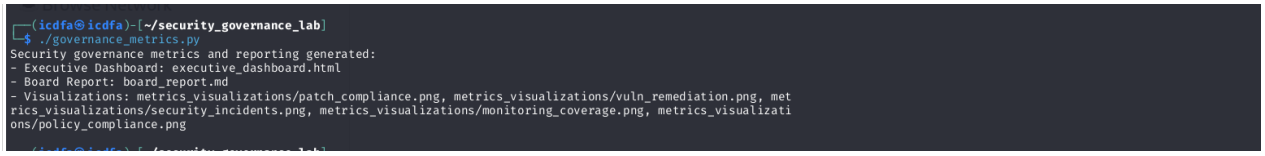


Figure 9 Security governance metrics script

2.

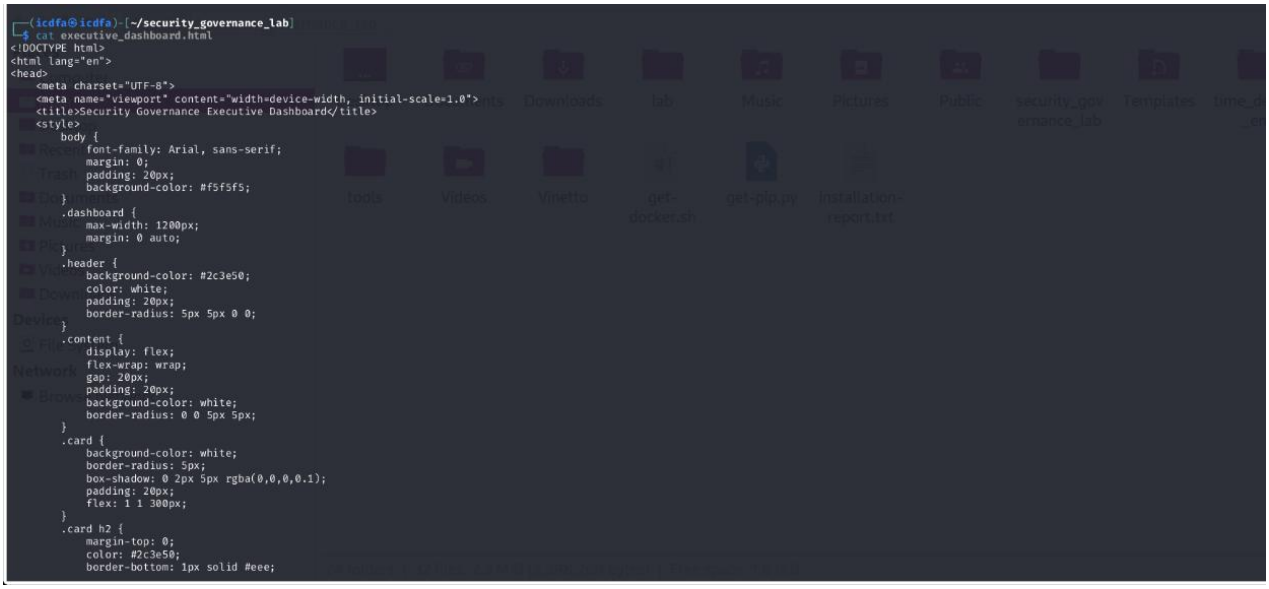


Figure 10 Executive dashboard HTML file

3.

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat board_report.md
# Security Governance Board Report

**Date: 2026-10-10**

## Executive Summary

This report provides an overview of the organization's security governance posture. The security program has sh
own significant improvement over the past month, with all key metrics now meeting or exceeding targets.

## Key Metrics

| Metric | Current | Target | Status |
|---|---|---|---|
| Patch Compliance | 100.0% | 95% | ✅ |
| Vulnerability Remediation Time | 1.0 days | 7 days | ✅ |
| Security Incidents | 0 | 0 | ✅ |
| Security Monitoring Coverage | 100.0% | 95% | ✅ |
| Policy Compliance | 100.0% | 95% | ✅ |

## Risk Summary

The organization's risk posture has improved significantly. There are currently:

- 0 Critical Risks
- 0 High Risks
- 2 Medium Risks
- 3 Low Risks

All identified risks have mitigation plans in place, with no risks accepted above the board-approved threshold.

## Security Incidents

There were no security incidents in the current reporting period. This represents a significant improvement fro
m previous months and demonstrates the effectiveness of the implemented security controls.

## Compliance Status

The organization is fully compliant with all applicable regulations and internal policies. Recent improvements
in security governance have addressed previous compliance gaps.

## Security Program Improvements
```

Figure 11 Board report

4.

```
cat: metrics_framework.md: No such file or directory

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat governance_metrics_framework.md
# Security Governance Metrics Framework

## Overview

This document outlines the security governance metrics framework implemented to support ongoing governance over
sight. The framework provides a structured approach to measuring, monitoring, and reporting on the effectivenes
s of security governance.

## Metrics Categories

The framework includes metrics in the following categories:

### 1. Implementation Metrics

Implementation metrics measure the extent to which security controls are implemented.

- **Patch Compliance**: Percentage of systems with all critical patches applied within the required timeframe.
- **Security Monitoring Coverage**: Percentage of critical systems covered by security monitoring.
- **Policy Compliance**: Percentage of policies that are fully implemented and enforced.

### 2. Effectiveness Metrics

Effectiveness metrics measure how well security controls are performing.

- **Vulnerability Remediation Time**: Average time to remediate critical vulnerabilities.
- **Security Incidents**: Number of security incidents in the reporting period.
- **Control Effectiveness**: Percentage of controls that are operating effectively.

### 3. Efficiency Metrics

Efficiency metrics measure the resources required to implement and maintain security controls.
```

Figure 12 Metrics framework document

5.

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat dashboard_user_guide.md
# Security Governance Dashboard User Guide

## Overview

This user guide provides instructions for using the Security Governance Dashboard. The dashboard is designed to
 provide executives and board members with a clear view of the organization's security governance posture.

## Accessing the Dashboard

The Security Governance Dashboard is available as an HTML file that can be opened in any web browser. To access
 the dashboard:

1. Open the `executive_dashboard.html` file in a web browser.
2. The dashboard will load automatically and display the current security governance metrics.

## Dashboard Sections

The dashboard is organized into the following sections:

### 1. Security Posture Summary

This section provides a high-level summary of the organization's security posture, including:

- Overall security posture rating
- Key metrics with current values
- Status indicators (green for good, yellow for warning, red for critical)

### 2. Risk Summary

This section provides an overview of the organization's risk posture, including:

- Number of risks by severity (critical, high, medium, low)
```

Figure 13 Dashboard user guide

6. A brief explanation (1-2 paragraphs) of how the metrics and reporting mechanisms support effective security governance

Security metrics and reporting mechanisms serve as the operational backbone of effective governance by converting raw technical telemetry into actionable intelligence for executive leadership. By tracking key risk and performance indicators such as patch compliance rates, remediation velocity, and monitoring coverage organizations gain measurable visibility into their posture. This quantitative insight eliminates oversight blind spots, aligns technical security operations with broader business risk tolerance, and ensures accountability across systems and control owners.

Furthermore, structured executive reporting artifacts (such as HTML dashboards and board reports) bridge the communication gap between technical teams and board members. Delivering clear, real-time summaries of security health enables leadership to make informed capital allocation decisions, validate regulatory compliance, and proactively enforce security mandates before policy gaps manifest as operational breaches.